CS422 — Computer Architecture

Assignment 1: Dynamic Binary Instrumentation with PIN

Group 8

Prem Kansagra (220816) Jenil Dobariya (220385) Poojal Katiyar (220770)
Raj Vinayak Meena (220861) Denchanadula Rushikesh Chary (220336)

Release: 04/02/2026 Deadline: 03/03/2026

Implementation Note. We instrumented all 13 SPEC 2006 ia32 benchmarks using a PIN tool built on the `TRACE_AddInstrumentFunction` API. Fast-forwarding is implemented with `InsertIfCall`/`InsertThenCall` using a cached boolean flag updated once per basic block (`UpdateAnalysisWindow`). Instruction counting is performed at BBL granularity via `BBL_InsertCall(... IncInsCountByN, IARG_UINT32, BBL_NumIns(bbl),...)` placed at the *end* of each BBL loop. Memory distribution arrays replace `std::map` for O(1) access; footprint sets use `std::unordered_set<UINT64>` storing 32-byte block indices. The fast-forward knob `-f N` is interpreted as $N \times 10^9$ instructions.

Part A — Instruction Mix

Part A: Instruction Type Counts & Percentages

Benchmark abbreviations: `PB`=perlbench, `BZ`=bzip2, `GCC`=gcc, `MCF`=mcf, `CAD`=cactusADM, `LS3`=leslie3d, `SPX`=soplex, `HMR`=hmmer, `LQ`=libquantum, `LBM`=lbm, `OMP`=omnetpp, `SP3`=sphinx3, `XLB`=xalancbmk.

Type	PB	BZ	GCC	MCF	CAD	LS3	SPX
Loads (32b)	357.0M	452.7M	338.9M	415.2M	1040.9M	736.4M	548.2M
Stores (32b)	205.4M	231.2M	170.3M	110.0M	331.3M	201.0M	97.6M
NOPs	947K	36.5K	3.9M	1.5M	2.3K	83.3K	3.6K
Direct Calls	12.7M	791.6K	30.2M	12.6M	531.6K	913	3.2M
Indirect Calls	2.9M	18	397.1K	0	193	17	111
Returns	15.5M	791.6K	30.6M	12.6M	531.8K	930	3.2M
Uncond. Branches	30.6M	21.3M	31.8M	8.3M	535.3K	169.2K	12.2M
Cond. Branches	129.9M	129.9M	145.8M	178.2M	4.3M	41.5M	103.4M
Logical Ops	99.9M	71.0M	54.6M	75.1M	1.6M	435.8K	14.3M
Rotate & Shift	4.3M	61.8M	634.9K	3.5M	1.1M	5.2M	10.7M
Flag Ops	856.7K	6.1K	0	0	5	32	22.5M
Vector Ins.	0	0	0	0	0	0	0
Cond. Moves	0	0	0	0	0	0	0
MMX & SSE	0	0	0	0	0	0	0
System Calls	0	0	0	0	0	0	0
Floating-Point	915.6K	0	0	0	851.7M	394.8M	309.8M
Others	701.4M	714.3M	702.0M	708.2M	139.7M	557.8M	520.7M
Total	1562M	1684M	1509M	1525M	2372M	1937M	1646M

Type	HMR	LQ	LBM	OMP	SP3	XLB
Loads (32b)	547.7M	279.9M	710.0M	371.2M	441.1M	365.6M
Stores (32b)	75.7M	129.2M	262.8M	229.8M	69.1M	167.4M
NOPs	34.3K	0	17	802.3K	179.7K	22.6M
Direct Calls	144.6K	556.7K	226	21.3M	3.3M	12.9M
Indirect Calls	937	0	57	3.7M	542	8.9M
Returns	145.5K	556.7K	283	25.0M	3.3M	21.9M
Uncond. Branches	205.8K	834.6K	4.8M	22.2M	5.5M	8.5M
Cond. Branches	144.4M	157.4M	7.8M	117.3M	112.8M	179.7M
Logical Ops	1.2M	146.3M	5.1M	60.0M	35.3M	37.2M
Rotate & Shift	294.1K	107.1M	196	7.1M	258.9K	5.5M
Flag Ops	5.6K	0	81.4K	20.2M	4.3M	1.6M
Vector Ins.	0	0	0	0	0	0
Cond. Moves	0	0	0	0	0	0
MMX & SSE	0	0	0	0	0	0
System Calls	0	0	0	0	0	0
Floating-Point	40.2K	0	965.3M	97.0M	412.4M	7.1M
Others	853.6M	587.3M	17.0M	625.4M	422.7M	693.6M
Total	1623M	1409M	1973M	1601M	1510M	1533M

Percentages for key instruction types:

Type (%)	PB	BZ	GCC	MCF	CAD	LS3	SPX	HMR	LQ	LBM	OMP	SP3	XLB
Loads	22.85	26.88	22.45	27.22	43.88	38.01	33.31	33.74	19.86	35.99	23.19	29.21	23.85
Stores	13.15	13.73	11.29	7.21	13.97	10.37	5.93	4.66	9.17	13.32	14.35	4.57	10.92
Cond.Br	8.32	7.72	9.66	11.69	0.18	2.14	6.29	8.89	11.17	0.40	7.33	7.47	11.72
FP	0.06	0.00	0.00	0.00	35.90	20.38	18.83	0.00	0.00	48.93	6.06	27.31	0.47
Others	44.89	42.42	46.52	46.43	5.89	28.79	31.64	52.58	41.68	0.86	39.06	27.99	45.26

Part B — CPI Estimates

Part B: Estimated CPI per Benchmark

CPI formula: $\text{CPI} = \dfrac{70 \times (\text{loads} + \text{stores}) + 1 \times (\text{all other instructions})}{\text{total instructions}}$

Benchmark	FF (B ins.)	Total Ins.	Window Ins.	CPI
400.perlbench	207	208B	1.000B	25.84
401.bzip2	301	302B	1.000B	29.02
403.gcc	107	108B	1.000B	24.28
429.mcf	377	378B	1.000B	24.76
436.cactusADM	584	585B	1.000B	40.91
437.leslie3d	2346	2347B	1.000B	34.39
450.soplex	364	365B	1.000B	28.07
456.hmmer	264	265B	1.000B	27.50
462.libquantum	3605	3606B	1.000B	21.03
470.lbm	830	831B	1.000B	35.02
471.omnetpp	43	44B	1.000B	26.90
482.sphinx3	1513	1514B	1.000B	24.31
483.xalancbmk	1331	1332B	1.000B	25.00

Observation. `436.cactusADM` has the highest CPI (40.91) due to its heavy floating-point and memory workload (44% loads + 14% stores + 36% FP). `470.lbm` is second (35.02) with ∼49% floating-point instructions. `462.libquantum` has the lowest CPI (21.03) as it is dominated by integer logical/shift operations (combined ∼18%) rather than memory-intensive operations.

Part C — Memory Footprint

Part C: Instruction & Data Footprints (32-byte block granularity)

Benchmark	Insn Blocks	Insn Bytes	Data Blocks	Data Bytes
400.perlbench	2,813	90,016 B	30,967	990,944 B
401.bzip2	780	24,960 B	2,528,517	80,912,544 B
403.gcc	242	7,744 B	66,117	2,115,744 B
429.mcf	65	2,080 B	11,672,601	373,523,232 B
436.cactusADM	1,224	39,168 B	4,442,969	142,175,008 B
437.leslie3d	2,631	84,192 B	2,476,189	79,238,048 B
450.soplex	645	20,640 B	5,658,614	181,075,648 B
456.hmmer	459	14,688 B	85,332	2,730,624 B
462.libquantum	67	2,144 B	1,048,592	33,554,944 B
470.lbm	413	13,216 B	13,121,298	419,881,536 B
471.omnetpp	890	28,480 B	528,914	16,925,248 B
482.sphinx3	1,193	38,176 B	153,969	4,927,008 B
483.xalancbmk	2,275	72,800 B	564,798	18,073,536 B

Observations. `470.lbm` and `429.mcf` have by far the largest data footprints (∼400 MB and ∼356 MB respectively), indicating large working sets that will stress cache hierarchies. `403.gcc` and `456.hmmer` have very compact data footprints (<3 MB). Instruction footprints are uniformly small (<91 KB), consistent with tight inner loops being the hot execution paths.

Single vs. Multi-Chunk Access Proportions:

Benchmark	Data Accesses		Insn Accesses	
	Single	Multi	Single	Multi
400.perlbench	99.9954%	0.0046%	92.1417%	7.8583%
401.bzip2	100.000%	0.0000%	90.2090%	9.7910%
403.gcc	100.000%	0.0000%	95.2043%	4.7957%
429.mcf	100.000%	0.0000%	94.6575%	5.3425%
436.cactusADM	100.000%	0.0000%	88.2447%	11.7553%
437.leslie3d	100.000%	0.0000%	92.1414%	7.8586%
450.soplex	98.9489%	1.0511%	93.5673%	6.4327%
456.hmmer	99.9998%	0.0002%	92.7513%	7.2487%
462.libquantum	100.000%	0.0000%	93.4133%	6.5867%
470.lbm	100.000%	0.0000%	93.1749%	6.8251%
471.omnetpp	99.1949%	0.8051%	95.8078%	4.1922%
482.sphinx3	100.000%	0.0000%	98.1082%	1.8918%
483.xalancbmk	99.6364%	0.3636%	96.9268%	3.0732%

Observations. Data accesses are overwhelmingly single-chunk (≥99% in all benchmarks), meaning nearly every memory operand fits within a single 32-byte block — consistent with naturally aligned scalar and small-vector accesses dominating ia32 workloads. `450.soplex` and `471.omnetpp` are slight outliers with ∼1% multi-chunk data accesses, likely due to unaligned or straddling struct field accesses. Instruction accesses show notably higher multi-chunk rates (2–12%), since instructions vary from 1–10 bytes and are not guaranteed to be 32-byte aligned, making boundary straddles far more common than for data. `436.cactusADM` has the highest instruction multi-chunk rate (11.8%), consistent with its heavy use of long 6-byte instructions visible in Part D.1. Note: three benchmarks (`cactusADM`, `470.lbm`, `437.leslie3d`) reported `nan` for the multi-chunk instruction percentage in earlier runs due to a floating-point division edge case in the tool; the correct values are now available from fresh runs and reported directly.

Part D — ISA Properties

Part D.1: Instruction Size Distribution (counts in millions)

Sz	PB	BZ	GCC	MCF	CAD	LS3	SPX	HMR	LQ	LBM	OMP	SP3	XLB
1	117.2	38.6	182.0	80.6	1.66	34.7	78.0	25.1	61.2	0.11	154.6	81.3	144.8
2	256.5	219.2	305.6	485.4	263.4	503.7	440.7	302.9	442.8	487.5	308.6	430.4	322.8
3	274.7	437.0	282.6	315.5	78.1	58.9	400.3	296.2	437.6	179.6	382.0	432.2	446.1
4	52.8	75.3	87.6	50.5	15.7	0.08	16.1	270.4	51.7	0.001	34.3	26.3	31.2
5	78.6	22.0	53.5	22.1	1.6	0.41	3.2	24.9	0.83	7.6	45.1	7.1	23.7
6	186.3	141.4	76.3	5.2	603.7	400.3	40.0	68.4	1.1	324.5	48.8	17.9	23.4
7	34.0	51.3	12.4	40.6	35.8	1.8	17.7	0.42	4.8	0.65	26.5	4.8	7.5
8	0.00	15.1	0	0	0	0	3.9	11.9	0	0	0	0.04	0.26
9	0	0	0	0	0	0	0	0	0	0	0	0	0.06
10	0.00	0.06	0	0	0	0.00	0	0	0	0.03	0.00	0.00	0.06

Part D.2-4: Operand & Register Count Distributions (counts in millions)

D.2 — Total operands per instruction:

Ops	PB	BZ	GCC	MCF	CAD	LS3	SPX	HMR	LQ	LBM	OMP	SP3	XLB
0	0.947	0.037	3.907	1.478	0.002	0.083	0.004	0.035	0	0.000	0.802	0.180	22.62
1	1.072	0.006	2.491	0	0.000	0.000	0.000	0.003	0	0.000	0.226	0.008	0.731
2	521.1	597.6	426.1	484.8	83.6	317.9	411.8	566.1	467.9	13.4	518.3	301.8	421.5
3	355.2	382.9	363.4	457.3	461.4	492.9	396.3	432.9	476.7	457.1	281.7	502.1	414.9
4	102.9	2.7	173.4	43.8	454.5	189.2	188.6	0.713	54.8	529.6	172.8	179.5	100.1
5	15.5	14.2	30.6	12.6	0.532	0.001	3.208	0.159	0.557	0.000	25.0	16.3	23.8
6	3.303	2.554	0	0	0.000	0.000	0	0.024	0	0.000	1.139	0.074	16.3

D.3 — Read register count per instruction:

RR	PB	BZ	GCC	MCF	CAD	LS3	SPX	HMR	LQ	LBM	OMP	SP3	XLB
2	537.1	533.6	482.0	677.5	532.1	653.9	579.9	562.4	461.1	604.4	540.4	585.9	456.9

D.4 — Write register count per instruction:

WR	PB	BZ	GCC	MCF	CAD	LS3	SPX	HMR	LQ	LBM	OMP	SP3	XLB
1	685.4	712.8	724.6	770.3	791.7	736.0	764.5	755.3	655.5	822.0	665.5	834.1	687.0

Part D.5-7: Memory Operand Distributions (counts in millions)

D.5 — Total memory operands per instruction:

Ops	PB	BZ	GCC	MCF	CAD	LS3	SPX	HMR	LQ	LBM	OMP	SP3	XLB
0	453.5	399.9	495.9	487.2	273.5	378.0	531.7	376.8	616.1	510.7	449.3	508.6	497.8
1	530.7	516.4	498.9	500.4	690.6	548.8	440.5	623.0	358.8	489.3	543.0	480.4	474.0
2	15.6	83.7	5.2	12.4	35.9	73.1	27.8	0.166	25.1	0.026	7.8	10.9	27.8
3	0	0	0	0	0	0	0	0	0	0	0	0	0

D.6 — Memory read operands per instruction:

Rds	PB	BZ	GCC	MCF	CAD	LS3	SPX	HMR	LQ	LBM	OMP	SP3	XLB
1	355.1	452.7	338.9	415.2	575.5	553.1	416.5	547.7	279.9	357.5	337.1	436.0	363.3

D.7 — Memory write operands per instruction:

Wrs	PB	BZ	GCC	MCF	CAD	LS3	SPX	HMR	LQ	LBM	OMP	SP3	XLB
0	794.8	768.8	829.7	890.0	813.2	858.0	920.4	924.3	870.8	868.2	780.9	933.7	833.2
1	205.0	231.2	170.3	110.0	186.8	142.0	79.6	75.7	129.2	131.8	219.1	66.3	166.3
2	0	0	0	0	0	0	0	0	0	0	0	0	0

Note on D5 & D7. The ia32 ISA physically cannot encode more than one memory address per instruction — the ModRM byte permits at most one memory operand location, yielding a maximum of 1 read + 1 write = 2 total memory operands per instruction. Consequently, **D5** (instructions with 3 memory operands) and **D7** (instructions with 2 memory write operands) are **zero across all 13 benchmarks**. This is not an instrumentation gap; the PIN tool correctly accumulates `memOpDistrib[numReadOps + numWriteOps]` and `memWriteOpDistrib[numWriteOps]` via `INS_InsertThenPredicatedCall` for every executed instruction, and the counts simply never reach those buckets because no such ia32 instruction exists. Note that **D7 wrs=1** reflects all instructions that *execute* a memory write (predicated count), which may differ slightly from the raw store count in Part A.

Part D.8-10: Memory Bytes, Immediates & Displacements

Benchmark	Max mem bytes/ins	Avg mem bytes/ins	Imm Min immediate value	Imm Max immediate value	Disp Min displacement	Disp Max displacement
perlbench	8	3.808	-2^{31}	$2^{31}-1$	−1408	135,918,104
bzip2	8	4.027	−858,993,459	1,431,655,766	−4848	135,000,192
gcc	8	3.808	−2,147,483,587	136,599,840	−96	138,633,083
mcf	8	4.097	−100,000,000	1,374,389,535	−76	134,957,120
cactusADM	8	7.555	-2^{31}	1,431,655,766	−2620	135,701,372
leslie3d	10	6.029	-2^{31}	$2^{31}-1$	−1760	135,182,404
soplex	8	5.516	−640,172,613	$2^{31}-1$	−344	135,855,532
hmmer	8	3.998	−987,654,321	$2^{31}-1$	−580	135,294,312
libquantum	8	3.866	−1	124	−64	134,982,404
lbm	8	7.952	−1,672,357,186	$2^{31}-1$	−16080	134,965,336
omnetpp	8	4.243	−2,092,037,281	$2^{31}-1$	−104	136,090,116
sphinx3	8	4.079	-2^{31}	$2^{31}-1$	−121088	135,127,364
xalancbmk	8	4.189	−1,431,655,765	$2^{31}-1$	−1392	139,657,912

Note: `cactusADM` now shows avg = 7.555 bytes/memory-instruction (previously `nan` in older runs). `leslie3d` uniquely shows a max of 10 bytes, indicating 80-bit x87 extended-precision FP memory accesses.

Benchmark Execution — `run_benchmarks.sh`. All 13 SPEC 2006 benchmarks were run on the CSE servers via a single Bash script that launches each PIN-instrumented process in the background (`&`), parallelising execution over SSH. The first 8 mandatory benchmarks (`perlbench`, `bzip2`, `gcc`, `mcf`, `soplex`, `hmmer`, `omnetpp`, `xalancbmk`) are launched together and a `wait` barrier ensures all complete before the 5 optional benchmarks (`cactusADM`, `leslie3d`, `libquantum`, `lbm`, `sphinx3`) are launched in a second parallel wave. Two benchmarks required special handling: `437.leslie3d` uses a `stdin` redirect (`< leslie3d.in`) outside the quoted command string, and `482.sphinx3` has a non-standard nested directory structure (`482.sphinx3/1/2/482.sphinx3/`). Each benchmark is invoked as:

```
pin -t HW1.so -o pin_final.out -f FF - ./binary args
```

where `-f FF` is the SimPoint fast-forward amount in billions. To replicate: place the compiled `HW1/` directory under `~/CS422/pin-dir/source/tools/`, then run `chmod +x run_benchmarks.sh && ./run_benchmarks.sh` from that directory.